Calibration-Free MXFP4 Recurrent States for Hybrid LLMs via Stochastic Rounding and Sinkhorn Balancing

Abstract

In hybrid language models built on Gated DeltaNet (GDN), each request carries a recurrent state whose size does not depend on the context. Serving systems keep this state in FP32, where it takes 151 MB per request in Qwen3.8-27B, far more than the KV cache of a typical chat request. We store the state in 4-bit MXFP4 as its only copy and without calibration. Like recent work, we round the state only once every 16 tokens and keep reads exact through a small buffer of recent updates. On top of this we add two techniques. First, we round the state stochastically under the standard MXFP4 block scale, a choice that follows from our analysis of the recurrence. Round-to-nearest is biased, so its errors pile up in heads that remember thousands of tokens, whereas stochastic rounding is unbiased except at the block maxima that the standard scale clips, and this slight clipping keeps it stable. Second, rows that share a block scale can differ widely in size, so before rounding we equalize them with power-of-two factors from Sinkhorn balancing. These factors are folded into the query and key and add no work at read time. We implement the method in vLLM with kernels for the AMD Instinct MI355X. On five Qwen hybrid models (9B–35B) and seven tasks with inputs of up to 128K tokens, the 4-bit state shows no significant accuracy loss in 30 of 35 model–task pairs, including all 14 of Qwen3.6-27B and Qwen3.8-27B, and the five losses fall on the longest reasoning answers of Qwen3.5. It shrinks the per-request state of Qwen3.8-27B from 151 MB to 33 MB and raises ShareGPT serving throughput by 18–22% at equal latency, and by 20–33% in multi-turn chat, where its smaller cache blocks let vLLM reuse far more of each prompt from the prefix cache.

1 Introduction

Gated DeltaNet (GDN) layers (Yang et al., 2025) summarize everything a request has read in a matrix of constant size, and hybrid models now use them for most of their layers. In Qwen3.8-27B (Qwen Team, 2026c), 48 of the 64 layers are GDN layers and only 16 keep a KV cache. Serving systems such as vLLM (Kwon et al., 2023) and SGLang (Zheng et al., 2024) keep this state in FP32, where it occupies 151 MB per request, as much as the FP8 KV cache of about 4,600 tokens (Figure 1a). Hybrid models thus move the memory problem of serving from the KV cache to the state. With an FP8 KV cache, a chat request of about 450 tokens spends 91% of its memory on the state, so the state decides how many users a GPU can serve, and compressing the KV cache to 4 bits (Chakrabarti et al., 2026) only raises its share. The state also limits prefix caching, which for hybrid models must keep a copy of the state for every cached prefix (Pan et al., 2025). vLLM reuses a cached state only at the boundary of a cache block and sizes the blocks to the state, 1,600 tokens

for the FP32 state with an FP8 KV cache, so in ShareGPT conversations at most 4% of prompt tokens come from the cache (Figure 1c).

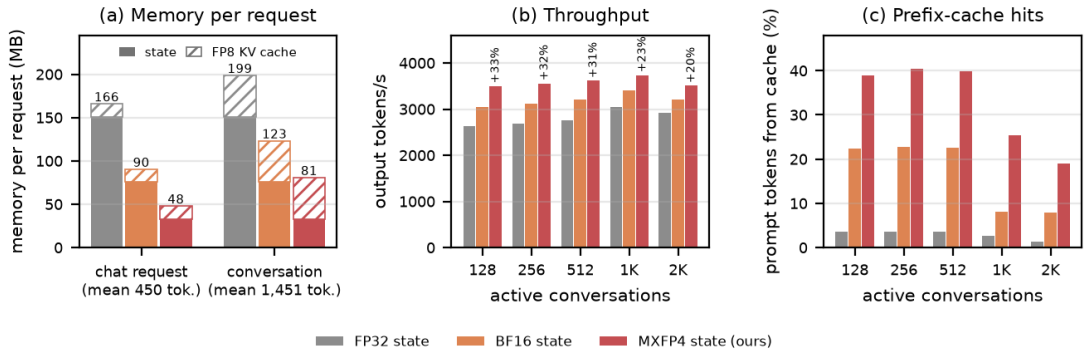


Figure 1. Multi-turn ShareGPT serving of Qwen3.8-27B on one MI355X with an FP8 KV cache, prefix caching and 128 concurrent clients. (a) Memory of one request, split into state and FP8 KV cache, at the mean length of a ShareGPT chat request and of a multi-turn conversation. (b) Output throughput. (c) Share of prompt tokens served from the prefix cache.

Storing the state in fewer bits is difficult because the recurrence passes every rounding error on to all later tokens, and a long reasoning answer is generated from tens of thousands of successive states. Calibrated methods give extra bits to channels or heads chosen offline; in this way STEPQuant (Yao et al., 2026) reports near-FP32 accuracy on Qwen3.8-27B at a 4-bit budget. Without calibration, LeapQuant (Pan et al., 2026) loses about 30 AIME points on Qwen3.5 at 4 bits, and ReplaySSM (Liou et al., 2026) keeps a second, high-precision copy of the state (Section 2).

This paper makes three contributions. First, it shows how the type of rounding error decides whether a low-bit state survives. Prior work observed that errors persist longer in long-memory heads and that rounding discards small updates, so that errors from successive roundings add up coherently (Yao et al., 2026; Pan et al., 2026, Appendix B.1; Li et al., 2026). We quantify this effect: the error that a biased rounding accumulates scales with the memory of a head, whereas unbiased noise scales only with its square root. We also show that stochastic rounding stays stable only under a block scale that clips slightly, such as the standard OCP scale (Section 3). Second, it turns this analysis into a 4-bit state that keeps a single MXFP4 copy, needs no calibration and adds no work at read time. The copy is rewritten once every 16 tokens, as in LeapQuant (Pan et al., 2026), by stochastic rounding, after power-of-two key-row factors from Sinkhorn balancing even out its rows. Third, it implements the state in vLLM on the AMD Instinct MI355X and evaluates it with paired significance tests and in serving (Section 4). On five Qwen hybrid models and seven tasks with inputs of up to 128K tokens, the state shows no significant loss in 30 of the 35 model–task pairs, including all 14 pairs of Qwen3.6-27B and Qwen3.8-27B, with the five losses on the longest answers of the Qwen3.5 models. It shrinks the per-request state of Qwen3.8-27B from 151 MB to 33 MB, runs the GDN decode kernel up to 5.2× faster than with the FP32 state and 2.6× faster than with a BF16 state, and raises ShareGPT throughput

by 18–22% at equal latency and by 31% at 1,024 concurrent requests. In multi-turn chat it serves 19–40% of prompt tokens from the prefix cache instead of at most 4% and raises throughput by 20–33%.

---

2 Related Work

Hybrid models pair a few attention layers with many recurrent layers whose memory does not grow with the context. Linear attention (Katharopoulos et al., 2020) first replaced the softmax with a state of fixed size, and state-space models such as Mamba (Gu & Dao, 2024) and Mamba-2 (Dao & Gu, 2024) made its decay depend on the input. GDN (Yang et al., 2025) writes to its state with the delta rule (Schlag et al., 2021; Yang et al., 2024), replacing the value stored under the current key rather than adding to it, and lets the state decay at a learned rate. Kimi Delta Attention gives every key channel its own decay (Kimi Team, 2025). Jamba (Lieber et al., 2024), Nemotron-H (NVIDIA, 2025), Kimi Linear (Kimi Team, 2025) and the Qwen models we study all keep some attention layers, because a state of constant size cannot recall arbitrary context exactly.

Most quantization methods for LLMs fit their parameters to calibration data, from GPTQ (Frantar et al., 2022) and SmoothQuant (Xiao et al., 2023) for weights and activations to KVQuant (Hooper et al., 2024) for the KV cache, while KIVI (Liu et al., 2024) needs no calibration and scales keys by channel and values by token. Keys and values are written once and then only read, whereas a recurrent state is rewritten at every token, so its rounding errors compound. Microscaling formats share one power-of-two scale across small blocks of values (Rouhani et al., 2023; Open Compute Project, 2023) and are supported in hardware by recent accelerators. SINQ (Müller et al., 2025) improves low-bit weight quantization without calibration by balancing each matrix along both axes with Sinkhorn-style iterations, an idea we transfer to the recurrent state. Stochastic rounding, which is unbiased in expectation, is an established tool in low-precision computing (Gupta et al., 2015; Croci et al., 2022), used for MXFP4 training (Tseng et al., 2025) and for the FP16 Mamba state of Nemotron 3 Super (NVIDIA, 2026). Quantized reasoning models tend to generate longer answers (Liu et al., 2025; Lotfi et al., 2026), which makes output length a useful diagnostic for a degraded state. For GDN layers, Kozyrev & Maiboroda (2026) show that 4-bit weights and activations are well tolerated, but they keep the recurrent state in high precision.

Work on the recurrent state itself is more recent. Earlier work on state-space models quantizes cached states with grouped or two-axis scales (Chiang et al., 2025; Chen et al., 2025). For delta-rule models, DAMP (Zhang et al., 2026) keeps a calibrated subset of key channels in higher precision, STEPQuant (Yao et al., 2026) allocates bits according to head lifetime and fits two-axis scales on calibration data, reporting near-FP32 accuracy on Qwen3.8-27B at a nominal 4-bit budget, and Chen & He (2026) allocate bits to state channels according to their decay without calibration. All of them round the state again after each token. LeapQuant (Pan et al., 2026) instead rounds the state once per window of

tokens, moves its largest outliers into a few rank-one FP16 terms, its compensator tokens, and scales the key rows before rounding. It matches FP32 at 8 bits but loses about 30 AIME points on Qwen3.5 at 4 bits. Concurrently, ReplaySSM (Liou et al., 2026) also rounds a per-window checkpoint stochastically, but because a shared 4-bit NVFP4 checkpoint loses 13 GPQA points, it keeps a separate FP16 or FP32 checkpoint for the recurrence. Two concurrent studies examine why the errors accumulate. Li et al. (2026) find that they are coherent across steps because small updates are lost in rounding, and they preserve these updates with a low-rank bypass buffer and a directional rounding rule. Jiang (2026) traces the failure of dynamic scaling to a feedback between the quantizer scale and the state from which it is computed. We adopt the window of LeapQuant but keep no high-precision copy of the state. LeapQuant found stochastic rounding ineffective at every token; we show that at the window boundary, under the standard OCP scale, it keeps a 4-bit MXFP4 state stable, and we extend the row smoothing to power-of-two Sinkhorn factors.

---

3 Method

We store the state of each head as a single MXFP4 matrix with E8M0 block scales and one power-of-two factor per key row, together with a small buffer of the updates from the current window of $C = 16$ tokens. Decoding reads the stored copy and the buffer, and at the end of each window a write-back folds the buffer into a new copy. Section 3.1 introduces the notation, Sections 3.2–3.4 motivate and describe the three design choices, and Appendix C gives the procedure as pseudocode.

3.1 Preliminaries

We describe one GDN head, since all heads are treated alike. Its state after t tokens is a matrix $S_t \in \mathbb{R}^{d_k \times d_v}$, and $d_k = d_v = 128$ in every model we use. Each token supplies a query q_t, a key k_t of unit norm, a value v_t, a decay $\alpha_t \in (0,1)$ and a step size $\beta_t \in (0,1)$, and the head updates its state and produces an output by (Yang et al., 2025)

$$\delta_t = \beta_t\,(v_t - \alpha_t S_{t-1}^{\top} k_t), \qquad S_t = \alpha_t S_{t-1} + k_t \delta_t^{\top}, \qquad h_t = S_t^{\top} q_t,$$

where we omit the constant scale $d_k^{-1/2}$ of q_t.

The state decays by α_t at each token and moves toward v_t along k_t by the step β_t. We quantize only S_t, which vLLM (Kwon et al., 2023) and SGLang (Zheng et al., 2024) store in FP32 by default and which occupies 151 MB per request in Qwen3.8-27B. Weights, activations, the small convolution state of GDN and the KV cache are left unchanged.

With the log-decay $g_t = \ln \alpha_t$, we define the memory of a head as $L = 1/\mathbb{E}[-g_t]$, the number of tokens after which a stored value has shrunk by a factor of about e. Setting the token-dependent part of the decay to zero gives L from the weights alone, and this estimate agrees with L measured on generated text (Spearman correlation 0.95). More

than half of the heads remember fewer than 64 tokens, but some remember thousands (Figure 2a), so a fixed window is short for some heads and long for others.

MXFP4 (Rouhani et al., 2023) stores blocks of 32 values, here 32 consecutive key rows of one value column, as 4-bit E2M1 elements from $\mathcal{G} = \{0, \pm 0.5, \pm 1, \pm 1.5, \pm 2, \pm 3, \pm 4, \pm 6\}$ with one shared power-of-two scale s, at 4.25 bits per element. The largest code is 6, so a block stores values up to $6s$. The Open Compute Project (OCP) rule (Open Compute Project, 2023) sets s from the block maximum m by $s = 2^{\lfloor \log_2 m \rfloor - 2}$. After scaling, the block maximum can exceed 6, in which case it is clipped to 6. A block x is stored as $\hat{x}_i = s\, Q(x_i/s)$. The rounding Q is either round-to-nearest (RTN) or stochastic rounding (SR) (Croci et al., 2022). SR sends a value y that lies between two neighboring elements of $\mathcal{G}$ to the upper one with probability equal to its distance from the lower one divided by the gap between them, so $\mathbb{E}[Q(y)] = y$ for $|y| \le 6$.

DAMP (Zhang et al., 2026) and STEPQuant (Yao et al., 2026) round the state again after each token. Following LeapQuant (Pan et al., 2026), we instead quantize once per window of C tokens. During a window the stored 4-bit state W is left untouched, and the new updates go to a small buffer. For token j of the window the buffer keeps k_j, δ_j and the log-decay since the window began, $\gamma_j = g_1 + \cdots + g_j$. The exact state is then the decayed stored state plus the decayed updates,

$$S_t = e^{\gamma_t} W + \sum_{j \le t} e^{\gamma_t - \gamma_j}\, k_j \delta_j^{\top},$$

so every read of the state is exact up to the BF16 storage of the buffer. The state is rounded once per window, when the write-back folds the buffer into a new W.

3.2 Stochastic Write-Back

Each write-back replaces the state with its rounded copy, so the error it introduces persists in every later window. Run the head twice on identical inputs, once from the stored state and once in FP32. The value v_t enters both runs of Equation (1) in the same way, so it drops out of their difference Δ_t, and only the part that acts on the state is left,

$$\Delta_t = \alpha_t \,(I - \beta_t k_t k_t^{\top})\, \Delta_{t-1}.$$

Two things shrink the error. The decay scales all of it by α_t, and the projection removes the share β_t of its component in the direction of k_t (Pan et al., 2026, Appendix B.1; Yao et al., 2026). As STEPQuant notes, only the decay acts on the part of the error that later keys seldom touch. Each write-back error therefore fades over the memory L of its head, and about L/C write-back errors overlap at any time. Unbiased errors of size σ partly cancel and accumulate to about $\sigma\sqrt{L/2C}$, whereas errors that share a bias b add up to about $b\, L/C$ (Appendix A, for $L \gg C$). At $C = 16$, L/C is 2 for a 32-token head and 256 for a 4,096-token head, so errors build up mainly in long-memory heads, consistent with STEPQuant (Yao et al., 2026), who find that errors persist longer in slowly decaying units. Beyond this, a bias is far more harmful than noise, and a longer window reduces both.

Figure 2b tests these predictions by teacher-forcing Qwen3.8-27B and Qwen3.5-9B on about 20 of their own AIME generations each, produced with the FP32 state (up to 41K tokens for Qwen3.8-27B and 78K for Qwen3.5-9B). We report the loss gap, the increase in mean per-token cross-entropy over the FP32 state averaged over traces, which Chen & He (2026) call the excess negative log-likelihood. With SR the loss gap roughly halves for every fourfold increase in C. RTN is almost four times worse than SR at $C = 16$ and about as bad as SR applied at every token.

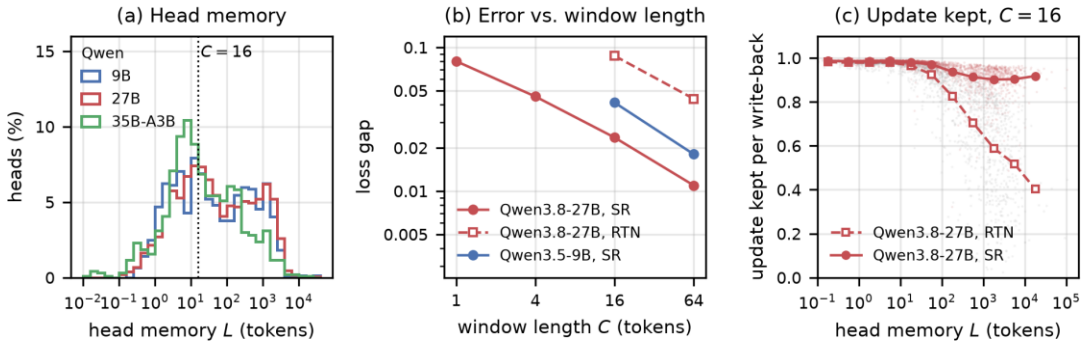


Figure 2. (a) Head memory estimated from the weights; the three 27B models coincide. (b) Loss gap versus window length, 20 traces of Qwen3.8-27B and 19 of Qwen3.5-9B. (c) Share of each window's update kept on three Qwen3.8-27B traces, per head (dots) and binned by memory (lines).

The bias arises in long-memory heads, whose state changes little within a window. RTN rounds most entries back to their previous values and loses the update, as noted for per-step quantization (Li et al., 2026; Pan et al., 2026, Appendix B.1), while SR preserves it on average. Figure 2c measures the kept share as the projection of the stored change onto the exact change on three Qwen3.8-27B traces. In heads that remember more than 2,048 tokens, RTN keeps about half and SR about 90%; SR without clipping keeps all of it, so the rest is lost to OCP clipping (Section 3.3). This loss degrades long generation. On the 60 AIME 2025 and 2026 problems, sampled once with a 40,960-token budget (half that of Table 1) and with a simulated write-back that keeps the state in FP32 between write-backs, RTN at $C = 16$ solves 32 problems on Qwen3.8-27B and none on Qwen3.5-9B and Qwen3.6-27B, where every generation reaches the budget. SR solves 55, 34 and 49, against 50, 42 and 48 with the FP32 state, and only the 9B difference approaches significance (McNemar $p = 0.057$).

Neither the window nor SR suffices alone. Windowed RTN is biased, and SR at every token has 3.4 times the loss gap of SR at $C = 16$ (Figure 2b). We therefore apply SR once per window with $C = 16$, since longer windows enlarge the buffer (Section 4.3). The write-back, and likewise the end of prefill, forms the exact state in FP32 registers and rounds it directly into the new MXFP4 copy, so no high-precision copy is kept.

3.3 Choice of Block Scale

The scale in Section 3.1 is the one we use. The question here is whether another power of two would be better. The OCP scale clips every entry above $6s$. The smallest scale that never clips is $s = 2^{\lceil \log_2(m/6) \rceil}$, since $6s$ is then just large enough to cover m. Stochastic rounding is exactly unbiased under that scale (Tseng et al., 2025), yet the scale is unstable. SR keeps the mean of each element but adds variance, so every write-back slightly inflates the expected squared norm of the state. In long-memory heads the decay over a window cannot offset this, so the error grows. Clipping under the OCP scale shrinks the largest elements and cancels that inflation.

Both rules belong to the family

$$s = 2^{\mathrm{round}(\log_2(\kappa\, m))},$$

with ties rounded up. A smaller κ clips more. The OCP rule of Section 3.1 is the point $\kappa = 2^{-2.5}$. The no-clip rule is the point $\kappa = \sqrt{2}/6$, except when $m/6$ is a power of two, where this κ picks twice the smallest no-clip scale.

Figure 3 sweeps κ on three models with SR at $C = 16$. Figure 3a shows the loss gap relative to the OCP rule. Without clipping it is 2–13 times higher, and a much smaller κ raises it again because too many elements are clipped. The best κ is about 0.19 on the two 27B models but 0.16–0.17 on Qwen3.5-35B-A3B, where $\kappa = 0.19$ is 21% worse than the OCP rule. The OCP rule is within 9% of the best κ on all three models without calibration, so we use it. Figure 3b counts the heads in which rounding inflates the state faster than it decays. Without clipping these are 2–12% of the heads, all with a memory above 1,700 tokens, against at most 0.1% under the OCP rule. Figure 3c shows the consequence on Qwen3.8-27B. Without clipping the loss gap keeps rising over the generation, while under the OCP rule it levels off.

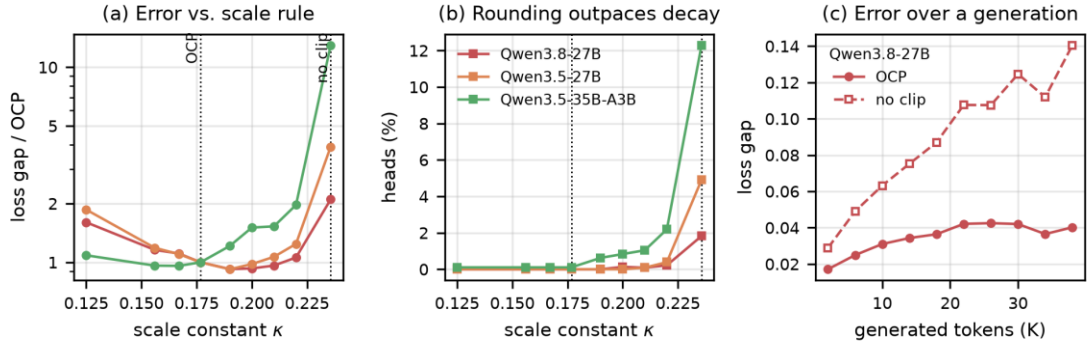


Figure 3. Block-scale rules of Equation (4), SR, $C = 16$, 20 traces per model. (a) Loss gap relative to the OCP rule. (b) Heads in which rounding inflates the state faster than it decays (3 traces). (c) Loss gap over the generation on Qwen3.8-27B.

3.4 Power-of-Two Key-Row Factors

An MXFP4 block spans 32 consecutive key rows of one value column, so its scale is set by the largest element in these rows. Key rows differ widely in magnitude (Figure 4a), and the elements of small rows are rounded on a grid that is coarse for them. At each write-back we therefore divide each key row i by a power-of-two factor λ_i and round the result to an MXFP4 matrix R. We store R and the vector λ, and the stored state is recovered as $W = \Lambda R$ with $\Lambda = \mathrm{diag}(\lambda)$. Factors on the value columns would have no effect, because a column factor scales whole blocks and their scales alike.

We compute λ by three rounds of Sinkhorn balancing (Sinkhorn & Knopp, 1967) on $|S|$. Each round divides every row, and then every column, by the square root of its mean magnitude, rounded to a power of two (Appendix B). A single pass of the row step without rounding is the smoothing rule of LeapQuant (Pan et al., 2026). The square root moves each step halfway toward balance and avoids overshooting. The column factors keep a few large columns from dominating the row statistics and are discarded afterwards. Balancing runs only in the write-back, once every C tokens, and its cost is included in the kernel times of Section 4.3.

The factors add no work at read time. Because $W^\top = R^\top \Lambda$, decoding multiplies q_t and k_t by λ, which is exact in BF16 since every factor is a power of two, and reads R unchanged. SINQ (Müller et al., 2025) also uses Sinkhorn scales, for weights. Our factors are powers of two, are recomputed at every write-back and act on a recurrent state.

Balancing reduces the median ratio of the largest to the median key row from 6.1 to 2.1 and lowers the loss gap by 38–54% on four models (Figure 4). Most of this gain comes from equalizing the rows at all: against the square-root row-mean rule of LeapQuant, rounded to powers of two, Sinkhorn balancing lowers the loss gap by a further 11% on Qwen3.8-27B (0.0165 to 0.0147, lower on 15 of 20 traces). It also outperforms single-pass factors with other fixed powers of the row means or row maxima and a Hadamard rotation along d_k (Ashkboos et al., 2024), and matches a costlier per-head choice of that power (Appendix B).

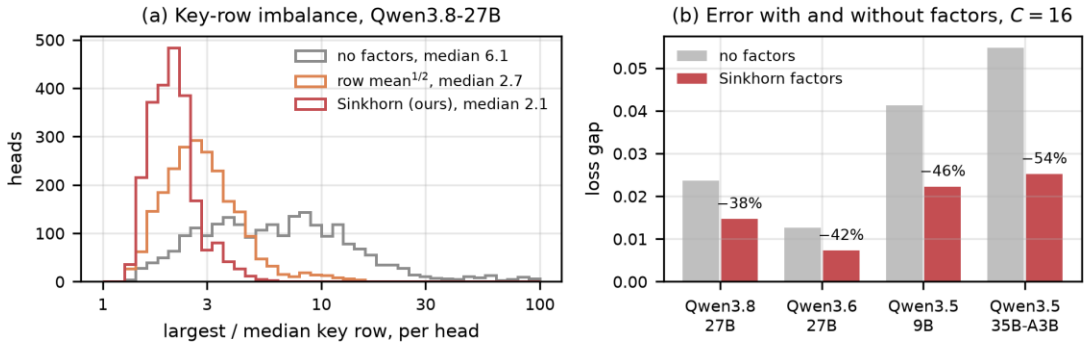


Figure 4. (a) Ratio of the largest to the median key row per head on Qwen3.8-27B, without factors, with single-pass row-mean factors and with Sinkhorn factors. (b) Loss gap at $C = 16$ without and with Sinkhorn factors.

4 Experiments

4.1 Setup

Setup. Our experiments use five Qwen hybrid models with GDN layers, Qwen3.5-9B, Qwen3.5-27B and Qwen3.5-35B-A3B (Qwen Team, 2026a), Qwen3.6-27B (Qwen Team, 2026b) and Qwen3.8-27B (Qwen Team, 2026c), with BF16 weights. Every experiment runs on a single AMD Instinct MI355X GPU. Accuracy runs use a BF16 KV cache, and the reference is the default FP32 state.

Implementation. We add our state to vLLM 0.23.1rc1 (Kwon et al., 2023) as a new state format, selected with `--mamba-ssm-cache-dtype mxfp4`, and keep all other layers in their baseline precision. Each GDN layer runs two kernels, both captured in the CUDA graph. The decode kernel, written in Triton (Tillet et al., 2019), reads the 4-bit state and the buffered updates of the current window with a single scaled matrix product and computes the output from Equation (2). The write-back kernel, written in Gluon (Triton Developers, 2025), the lower-level dialect of Triton, runs only when a window ends. It rebuilds the state, computes the key-row factors and block scales and rounds the result with the SR conversion instruction of the MI355X (AMD, 2025). Prefix caching stores and restores the 4-bit state pages without conversion. We use windows of $C = 16$ tokens, the OCP block scale and three Sinkhorn rounds throughout.

Tasks. The tasks are GSM8K (Cobbe et al., 2021, 1,319 problems), MMLU-Pro (Wang et al., 2024, 50 questions from each of its 14 categories), LongCodeQA (Rando et al., 2025, 113 questions at 32K and 92 at 128K input tokens), GPQA-Diamond (Rein et al., 2024, 198 questions), LiveCodeBench (Jain et al., 2025, the 175 problems added in release v6) and AIME 2025 and 2026 (Balunović et al., 2025, 60 problems). We use the thinking-mode sampling settings of each model card with at most 81,920 output tokens, and sample AIME six times and LiveCodeBench three times per problem. Both states use the same seeds, and we test each paired difference at $p < 0.05$ (Appendix D).

4.2 Accuracy

Model	State	GSM8K	MMLU-Pro	LCQA 32K	LCQA 128K	GPQA-D	LCB	AIME
Qwen3.5-9B	FP32	96.9	79.9	72.6	70.7	80.6	66.9	87.2
	Ours	96.4	79.4	70.8	71.7	77.8	**57.1**	**83.1**
Qwen3.5-27B	FP32	96.7	83.7	76.1	68.5	87.4	81.9	92.5
	Ours	97.4	84.7	74.3	69.6	86.4	**77.9**	90.6
Qwen3.5-35B-A3B	FP32	96.6	83.4	75.2	75.0	84.3	74.5	92.8
	Ours	96.6	83.4	76.1	71.7	84.8	**65.5**	**84.4**
Qwen3.6-27B	FP32	95.9	83.6	77.9	78.6	87.4	82.3	93.9
	Ours	95.2	82.6	77.0	77.5	84.3	81.0	93.6

Model	State	GSM8K	MMLU-Pro	LCQA 32K	LCQA 128K	GPQA-D	LCB	AIME
Qwen3.8-27B	FP32	97.2	82.4	79.6	78.3	88.9	83.2	96.4
	Ours	97.1	81.1	81.4	77.2	88.4	84.2	96.4

Table 1. Accuracy (%) with the FP32 state and our MXFP4 state. Bold marks a significant loss. LCQA is LongCodeQA and LCB is LiveCodeBench. Appendix D gives the p-values.

Our state shows no significant loss on 30 of the 35 model–task pairs (Table 1), including all 14 pairs of Qwen3.6-27B and Qwen3.8-27B, although on Qwen3.6-27B the differences are small losses of 1.2 points on average. The five losses are all on Qwen3.5 models and on the two tasks with the longest answers, LiveCodeBench and AIME, where these models generate 27K–36K tokens per answer. Long inputs cause no loss. LongCodeQA with 128K input tokens shows no significant change on any model, since prefill rounds the state far less often than decoding.

Long generation on Qwen3.5 is fragile even with a 16-bit state (Figure 5). Rounding the state to BF16 after every token, the low-precision option of vLLM, loses more than our state on Qwen3.5-9B and Qwen3.5-27B, up to 22 points on AIME, and less on Qwen3.5-35B-A3B. Almost all wrong AIME answers with the BF16 state stop unfinished at the 81,920-token limit, an overlong generation also seen in quantized reasoning models (Liu et al., 2025; Lotfi et al., 2026), whereas our state keeps the mean AIME output length within 10% of FP32 on all five models.

Figure 6 separates the two techniques. With the same 16-token window and no key-row factors, RTN loses 21 points on LiveCodeBench and 24 on AIME for Qwen3.8-27B, while SR alone shows no loss. The Sinkhorn factors make no significant difference on this model, but on Qwen3.5-35B-A3B they recover 7.2 of the 16.2 LiveCodeBench points that SR alone loses ($p < 0.001$) and 3.6 of 11.9 AIME points. The RTN and SR runs simulate the write-back with the state in FP32 between write-backs, while our state runs the native kernel with its BF16 buffer.

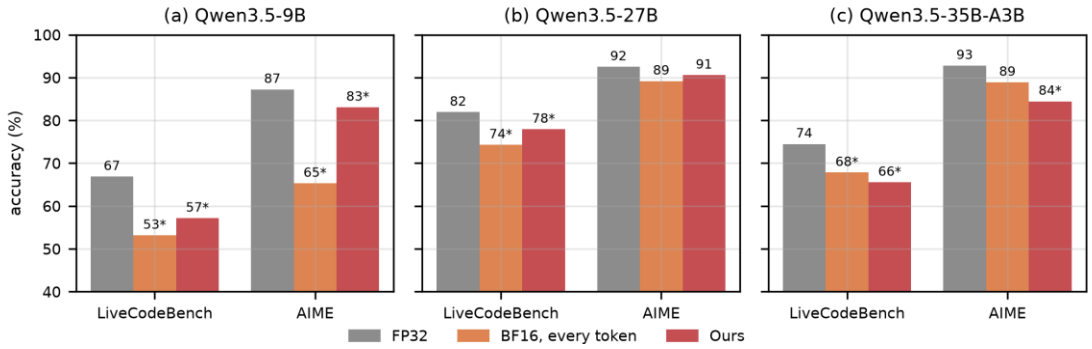


Figure 5. Accuracy with a BF16 state rounded after every token. An asterisk marks a significant loss against FP32 (Appendix D).

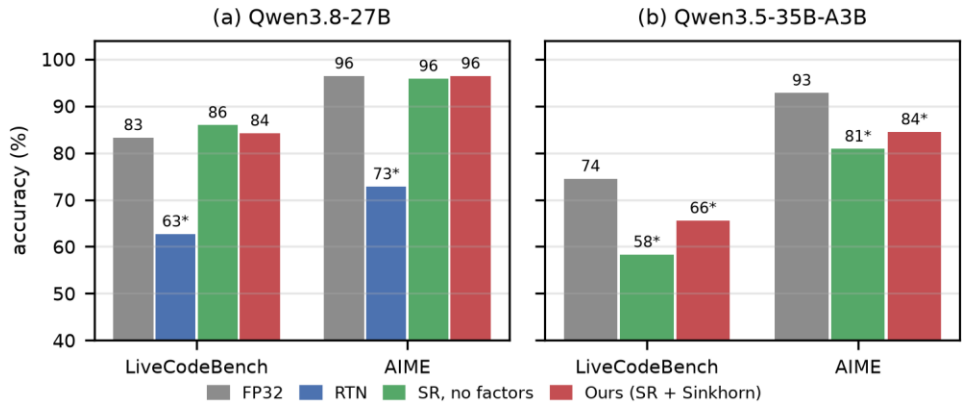


Figure 6. Ablation of the write-back with $C = 16$. RTN and SR without key-row factors, both with a simulated write-back, against our state on the native kernel. Panel (b) has no RTN run. An asterisk marks a significant loss against FP32.

4.3 Efficiency

Our state shrinks the per-request state of Qwen3.8-27B 4.6-fold and makes the GDN decode kernel up to 5.2× faster. With an FP8 KV cache and requests of 8,192 tokens, vLLM's startup estimate of the number of concurrent requests that fit in the cache pool rises from 443 to 665, a factor of 1.5. The stored state takes 4.31 bits per element, or 7.0–7.3 bits with the 16-token buffer, so a request of Qwen3.8-27B holds 33 MB of state instead of 151 MB.

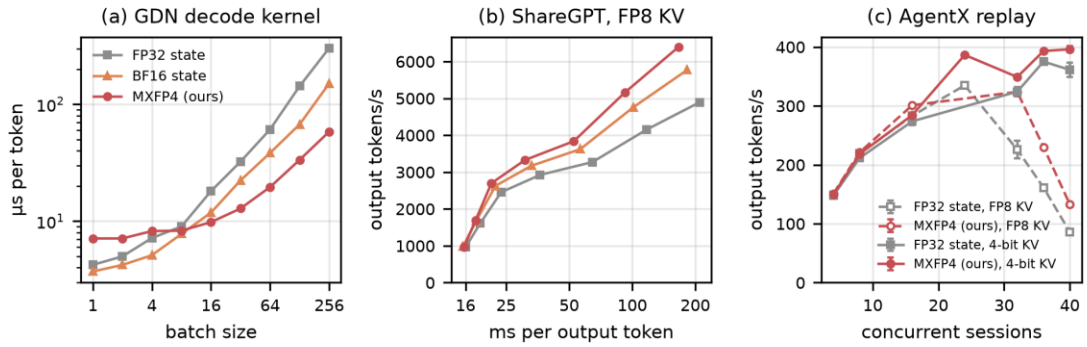


Figure 7. Decoding on one MI355X. (a) GDN kernel time per token, one Qwen3.6-27B layer. (b) ShareGPT serving of Qwen3.8-27B. (c) Replay of agentic sessions with Qwen3.8-27B. Open markers use an FP8 KV cache.

Kernel. From batch 16 to 256 our GDN kernel is 1.8–5.2× faster than the FP32 kernel of vLLM (Figure 7a, one GDN layer of Qwen3.6-27B, which has the same shape as in Qwen3.8-27B), and 1.2× and 2.6× faster than the BF16 kernel at batch 16 and 256. The speedup over FP32 exceeds the 4.6-fold reduction in state size because the FP32 kernel reads and writes the whole state at every token, whereas ours mostly reads it and rewrites it only once every 16 tokens. A full decoding step at batch 256 is 1.13× faster on Qwen3.5-

9B and 1.17× on Qwen3.6-27B, with 3,584 prompt and 512 output tokens per request, following the protocol of Pan et al. (2026). At batch 4 and below the kernel is slower, since the buffered updates add fixed work while memory traffic is small.

Single-turn serving. On ShareGPT (Chiang et al., 2023) with Qwen3.8-27B, an FP8 KV cache and prefix caching off, our state delivers 18–22% more output throughput than the FP32 state at equal time per output token and 31% more at the highest load of 1,024 concurrent requests (Figure 7b). Against a BF16 state, which loses accuracy on the Qwen3.5 models (Figure 5), the gain at equal time per output token is 6–8%. It needs less than a third of the cache memory of the FP32 state at every load, and at 1,024 requests the FP32 state fills the 195 GiB pool while ours uses 54 GiB (Figure 8a).

Multi-turn serving. Figures 1b and 1c replay 2,048 ShareGPT conversations with 128 clients, prefix caching and an FP8 KV cache. vLLM caches the state only at the end of each cache block, and our smaller state shrinks these blocks from 1,600 to 384 tokens. Our state therefore serves 19–40% of prompt tokens from the cache instead of at most 4%, and delivers 20–33% more output throughput than the FP32 state and 10–15% more than a BF16 state.

Agentic sessions. Figure 7c replays the coding-agent traces of the AgentX scenario of InferenceX (SemiAnalysis, 2025, 2026), with 94K input tokens per request and prefix caching. With the 4-bit KV cache of UltraQuant (Chakrabarti et al., 2026), our state raises throughput by 4–10% from 16 to 40 sessions. With an FP8 KV cache, both states slow down once the cache pool fills, ours later than the FP32 state, and at 40 sessions ours delivers 55% more throughput in a single run.

At 94K tokens per request the KV cache dominates the memory of a request, so our state saves only 4% of it, against 71% on ShareGPT (Figure 8b). Short-context serving therefore gains the most.

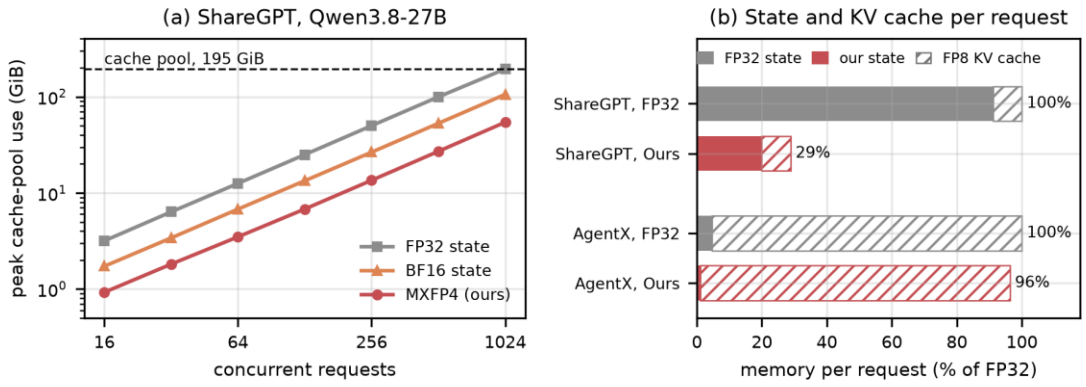


Figure 8. Memory in serving with Qwen3.8-27B. (a) Peak cache-pool use on ShareGPT with an FP8 KV cache. (b) Memory per request with an FP8 KV cache, from the state and cache layouts at the mean ShareGPT and AgentX lengths.

5 Conclusion

Hybrid models shift the memory cost of serving from the KV cache to the recurrent state. We keep this state in MXFP4 as a single copy, with no calibration. Stochastic rounding under the standard OCP block scale keeps the rounding errors from adding up coherently, and in our measurements its slight clipping keeps the state from inflating; rounding once per window with balanced key rows reduces the error that remains. On five Qwen hybrid models the resulting state shows no significant loss in 30 of the 35 model–task pairs, and in vLLM on the MI355X it shrinks the per-request state of Qwen3.8-27B 4.6-fold and turns the saving into serving capacity and throughput. As KV caches move to 4 bits, the precision of the state becomes the next lever for serving hybrid models at scale.

References

- AMD. CDNA4 instruction set architecture reference guide. https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/instruction-set-architectures/amd-instinct-cdna4-instruction-set-architecture.pdf, 2025.
- Ashkboos, S., Mohtashami, A., Croci, M. L., Li, B., Cameron, P., Jaggi, M., Alistarh, D., Hoefler, T., and Hensman, J. QuaRot: Outlier-free 4-bit inference in rotated LLMs. In *NeurIPS*, 2024.
- Balunović, M., Dekoninck, J., Petrov, I., Jovanović, N., and Vechev, M. MathArena: Evaluating LLMs on uncontaminated math competitions. arXiv:2505.23281, 2025.
- Chakrabarti, I., Limpus, D., et al. UltraQuant: 4-bit KV caching for context-heavy agents. arXiv:2606.20474, 2026.
- Chen, H. and He, J. Low-bit recurrent states in hybrid language models. arXiv:2609.30950, 2026.
- Chen, T., Chen, Y., Wang, P., Xu, W., Zhu, Z., and Cheng, J. Q-Mamba: Towards more efficient Mamba models via post-training quantization. In *Findings of ACL*, 2025.
- Chiang, H., Chang, C., Frumkin, N., Wu, K., Abdelfattah, M. S., and Marculescu, D. Quamba2: A robust and scalable post-training quantization framework for selective state space models. In *ICML*, 2025.
- Chiang, W.-L., Li, Z., Lin, Z., Sheng, Y., Wu, Z., Zhang, H., Zheng, L., Zhuang, S., Zhuang, Y., Gonzalez, J. E., Stoica, I., and Xing, E. P. Vicuna: An open-source chatbot impressing GPT-4 with 90%* ChatGPT quality. LMSYS blog, https://lmsys.org/blog/2023-03-30-vicuna/, 2023.
- Cobbe, K., Kosaraju, V., Bavarian, M., et al. Training verifiers to solve math word problems. arXiv:2110.14168, 2021.
- Croci, M., Fasi, M., Higham, N. J., Mary, T., and Mikaitis, M. Stochastic rounding: Implementation, error analysis and applications. *Royal Society Open Science*, 9(3):211631, 2022.

- Dao, T. and Gu, A. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *ICML*, 2024.
- Fisher, R. A. *The Design of Experiments*. Oliver and Boyd, 1935.
- Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. GPTQ: Accurate post-training quantization for generative pre-trained transformers. arXiv:2210.17323, 2022.
- Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. In *COLM*, 2024.
- Gupta, S., Agrawal, A., Gopalakrishnan, K., and Narayanan, P. Deep learning with limited numerical precision. In *ICML*, 2015.
- Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M. W., Shao, Y. S., Keutzer, K., and Gholami, A. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *NeurIPS*, 2024.
- Jain, N., Han, K., Gu, A., et al. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *ICLR*, 2025.
- Jiang, Y. When dynamic scaling fails: A scale–state feedback governs recurrent-state quantization in linear-attention and SSM LLMs. arXiv:2610.15139, 2026.
- Katharopoulos, A., Vyas, A., Pappas, N., and Fleuret, F. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *ICML*, 2020.
- Kimi Team. Kimi Linear: An expressive, efficient attention architecture. arXiv:2510.26692, 2025.
- Kozyrev, S. and Maiboroda, D. Why Gated DeltaNet survives 4-bit quantization: NVFP4 W4A4 for the recurrent half of a hybrid 27B LLM. arXiv:2609.04098, 2026.
- Kwon, W., Li, Z., Zhuang, S., et al. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023.
- Li, H., Dong, L., Xiao, K., and Xing, G. Preserving updates in low-bit recurrent state quantization. arXiv:2610.15709, 2026.
- Lieber, O., Lenz, B., Bata, H., et al. Jamba: A hybrid Transformer-Mamba language model. arXiv:2403.19887, 2024.
- Liou, Z.-W., Nover, H., Naik, A., Skliar, A., Shovkun, I., Shahaf, I., and Dao, T. ReplaySSM: Cache SSM inputs, not state. arXiv:2610.13778, 2026.
- Liu, R., Sun, Y., Zhang, M., Bai, H., Yu, X., Yu, T., Yuan, C., and Hou, L. Quantization hurts reasoning? An empirical study on quantized reasoning models. arXiv:2504.04823, 2025.
- Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., and Hu, X. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *ICML*, 2024.
- Lotfi, S., Kirichenko, P., Li, S., and Liu, Z. Quantized reasoning models think they need to think longer, but they do not. arXiv:2606.00206, 2026.
- McNemar, Q. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

- Müller, L. K., Bich, P., Zhuang, J., Celik, A., Benfenati, L., and Cavigelli, L. SINQ: Sinkhorn-normalized quantization for calibration-free low-precision LLM weights. arXiv:2509.22944, 2025.
- NVIDIA. Nemotron-H: A family of accurate and efficient hybrid Mamba-Transformer models. arXiv:2504.03624, 2025.
- NVIDIA. Nemotron 3 Super: Open, efficient mixture-of-experts hybrid Mamba-Transformer model for agentic reasoning. Technical report, 2026.
- Open Compute Project. OCP Microscaling Formats (MX) Specification, version 1.0, 2023.
- Pan, R., Wang, Z., Jia, Z., Karakus, C., Zancato, L., Dao, T., Wang, Y., and Netravali, R. Marconi: Prefix caching for the era of hybrid LLMs. In *MLSys*, 2025.
- Pan, Y., Xi, H., et al. LeapQuant: Efficient linear attention with accurate recurrent state quantization. arXiv:2609.38166, 2026.
- Qwen Team. Qwen3.5: Towards native multimodal agents. https://qwen.ai/blog?id=qwen3.5, 2026a.
- Qwen Team. Qwen3.6-27B: Flagship-level coding in a 27B dense model. https://qwen.ai/blog?id=qwen3.6-27b, 2026b.
- Qwen Team. Qwen3.8-Max: A new bar for coding and cowork. https://qwen.ai/blog?id=qwen3.8, 2026c.
- Rando, S., Kyuragi, Y., Romani, L., Galasso, F., et al. LongCodeBench: Evaluating coding LLMs at 1M context windows. arXiv:2505.07897, 2025.
- Rein, D., Hou, B. L., Stickland, A. C., et al. GPQA: A graduate-level Google-proof Q&A benchmark. In *COLM*, 2024.
- Rouhani, B. D., et al. Microscaling data formats for deep learning. arXiv:2310.10537, 2023.
- SemiAnalysis. InferenceX: Open-source continuous inference benchmark. https://github.com/SemiAnalysisAI/InferenceX, 2025.
- SemiAnalysis. cc-traces-weka-062126-256k: Coding-agent session traces. https://huggingface.co/datasets/semianalysisai/cc-traces-weka-062126-256k, 2026.
- Schlag, I., Irie, K., and Schmidhuber, J. Linear transformers are secretly fast weight programmers. In *ICML*, 2021.
- Sinkhorn, R. and Knopp, P. Concerning nonnegative matrices and doubly stochastic matrices. *Pacific Journal of Mathematics*, 21(2):343–348, 1967.
- Tillet, P., Kung, H. T., and Cox, D. Triton: An intermediate language and compiler for tiled neural network computations. In *MAPL*, 2019.
- Triton Developers. Gluon, a lower-level GPU programming language in Triton. https://triton-lang.org/main/gluon/, 2025.
- Tseng, A., Yu, T., and Park, Y. Training LLMs with MXFP4. In *AISTATS*, 2025.

- Wang, Y., Ma, X., Zhang, G., et al. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. In *NeurIPS Datasets and Benchmarks*, 2024.
- Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., and Han, S. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *ICML*, 2023.
- Yang, S., Kautz, J., and Hatamizadeh, A. Gated delta networks: Improving Mamba2 with delta rule. In *ICLR*, 2025.
- Yang, S., Wang, B., Zhang, Y., Shen, Y., and Kim, Y. Parallelizing linear transformers with the delta rule over sequence length. In *NeurIPS*, 2024.
- Yao, B., Xu, H., Lin, H., et al. STEPQuant: When and where errors matter in delta-rule recurrent state quantization. arXiv:2609.38169, 2026.
- Zhang, T., Tan, J., et al. DAMP: Decay-aware mixed-precision recurrent-state quantization. arXiv:2608.27513, 2026.
- Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. SGLang: Efficient execution of structured language model programs. In *NeurIPS*, 2024.

Appendix A. Error Growth Across Write-Backs

Each write-back adds a rounding error, and later tokens shrink it. By Equation (3), a token multiplies the error by $\alpha_t(I - \beta_t k_t k_t^\top)$. The factor $I - \beta_t k_t k_t^\top$ only removes part of the error along k_t and never enlarges it, so over a window of C tokens an error keeps at most the fraction $\rho = e^{-C/L}$ of its size that the decay leaves. The error after many write-backs is therefore at most the newest one, plus ρ times the one before, plus ρ^2 times the one before that, and so on. Random errors of size σ, as from SR, partly cancel, so their squares add. Errors that share a bias b, as from RTN, add directly,

$$\text{unbiased} \quad \sigma\sqrt{1 + \rho^2 + \rho^4 + \cdots} \approx \sigma\sqrt{\frac{L}{2C}}, \qquad \text{biased} \quad b\,(1 + \rho + \rho^2 + \cdots) \approx b\,\frac{L}{C},$$

using $1 - \rho \approx C/L$ for $L \gg C$. For the same error per write-back, a bias thus grows about $\sqrt{2L/C}$ times larger, 23 times for $L = 4{,}096$. The argument assumes the same inputs with and without rounding (Section 3.2), the same decay in every window and, for SR, independent errors, which holds except at the block maxima that the OCP scale clips (Section 3.3).

Appendix B. Key-Row Factor Rules

Table B.1 compares ways to set the key-row factors of Section 3.4. All use the OCP block scale, SR and $C = 16$, and recompute the factors at every write-back.

- *Single-pass rules* divide each key row by the power of two nearest to its mean or maximum magnitude. The square-root versions take the square root first, which removes half of the imbalance between rows. The square-root row-mean rule is the row smoothing of LeapQuant (Pan et al., 2026), here rounded to a power of two.
- The *per-head rule* tries the powers 0, 1/4, 1/2, 3/4 and 1 of the row means and keeps, for each head, the one with the smallest expected rounding error.
- *Sinkhorn balancing* (ours) starts from $\lambda = \mu = 1$ and alternates three times between the row and column updates
  $$\lambda_i \leftarrow \lambda_i \, P\left[(\mathrm{mean}_j |S_{ij}|/(\lambda_i \mu_j))^{1/2}\right], \qquad \mu_j \leftarrow \mu_j \, P\left[(\mathrm{mean}_i |S_{ij}|/(\lambda_i \mu_j))^{1/2}\right],$$
  where $P(y) = 2^{\mathrm{round}(\log_2 y)}$, and discards the column factors μ.
- The last row replaces the factors by a Hadamard rotation along d_k, as in QuaRot (Ashkboos et al., 2024).

All rules are evaluated on the 20 Qwen3.8-27B generations of Figure 2b. Sinkhorn balancing has the lowest loss gap. It beats the single-pass rules with a fixed power on 15–17 of the 20 traces and is about as good as the per-head rule, which must evaluate five candidate factors for every head.

Rule (Qwen3.8-27B, $C = 16$, SR)	Loss gap
No factors	0.0236
Row maximum, square root	0.0179
Row mean, full	0.0162
Row mean, square root	0.0165
Row mean, power chosen per head	0.0151
Sinkhorn (ours)	0.0147
Hadamard rotation, no factors	0.0712

Table B.1. Mean loss gap of key-row factor rules on Qwen3.8-27B, 20 traces. Lower is better.

Appendix C. Decoding Procedure

Algorithm C.1 lists one decoding step of a head. Prefill forms the exact final state and ends with steps 6–8.

Algorithm C.1. One decoding step of a GDN head.

Notation. t is the position in the current window of C tokens. R is the MXFP4 matrix with its E8M0 block scales and λ the key-row factors, so the stored state is $W = \Lambda R$. The buffer holds k_j, δ_j and γ_j for the earlier tokens j of the window, where δ_j is the update and $\gamma_j = \sum_{i \le j} g_i$ the log-decay since the window began. S_t is the exact state of Equation (2).

Inputs. W, the buffer, and the query q_t, key k_t, value v_t, step size β_t and log-decay g_t.

1. Compute $W^\top q_t$ and $W^\top k_t$ as one MXFP4 product, $R^\top[\lambda \odot q_t,\ \lambda \odot k_t]$.
2. Combine $W^\top k_t$ with the buffer to obtain $S_{t-1}^\top k_t$ (Equation 2).
3. Compute δ_t (Equation 1) and add k_t, δ_t and γ_t to the buffer.
4. Emit the output $h_t = S_t^\top q_t$, obtained from $W^\top q_t$ and the buffer in the same way.
5. If $t < C$, stop. Otherwise form S_C in FP32 (Equation 2).
6. Compute the power-of-two key-row factors λ of S_C by Sinkhorn balancing and divide each key row by its factor (Section 3.4).
7. Split each value column into blocks of 32 key rows, set each block scale by the OCP rule and round with SR (Sections 3.1 and 3.3).
8. Store the result as the new R and λ, and clear the buffer.

Output. h_t, and every C tokens a new stored state W.

Appendix D. Paired Test Results

Both states receive the same prompts and seeds, so each item yields a pair of outcomes. When a problem is sampled once we use an exact McNemar test (McNemar, 1947). It counts the items that only the FP32 state solves and those that only our state solves. Under the null hypothesis of equal accuracy, each such item falls on either side with probability 1/2, and p is the binomial probability of a split at least as uneven as the one observed. When a problem is sampled several times its samples are not independent, so we use a sign-flip test over problems (Fisher, 1935). For each problem we take the number of its samples that our state solves minus the number that the FP32 state solves. If the two states were equally accurate, each of these values would be equally likely to be positive or negative. We therefore flip their signs at random 20,000 times on the stored results, and p is the share of flips whose sum is at least as far from zero as the observed sum. Both tests are two-sided, and we call a difference significant when $p < 0.05$.

Figure D.1 shows the change in accuracy for every cell of Table 1. Our state loses significantly only on LiveCodeBench and AIME for the Qwen3.5 models, and the one other significant difference is a gain of 0.7 points on GSM8K for Qwen3.5-27B.

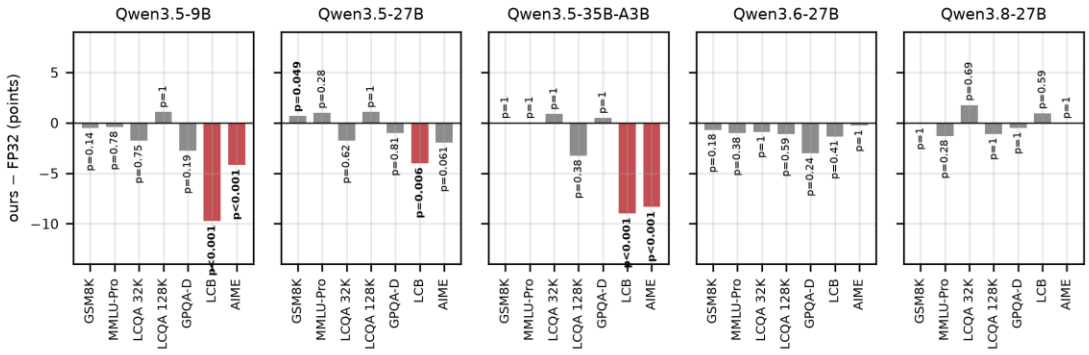


Figure D.1. Accuracy of our state minus that of the FP32 state. Each bar is labeled with its p-value, and red bars are significant losses.

Appendix E. Accuracy Retention Reported by Related Methods

No implementation of LeapQuant (Pan et al., 2026) or STEPQuant (Yao et al., 2026) was available to us, and the three studies differ in hardware, serving system, AIME problem set, number of samples and output budget. We therefore take their published numbers and normalize each method by the FP32 baseline of its own study, as STEPQuant does when comparing with DAMP. Table E.1 reports this retention on the two tasks that all three studies share, AIME and LiveCodeBench v6. The comparison is indicative, not controlled: no difference in it is tested for significance.

Method	Calibration	Model	FP32 (AIME / LCB)	Method (AIME / LCB)	Retention
LeapQuant, 4-bit	no	Qwen3.5-9B	87.9 / 64.1	58.1 / 34.0	60.6%
Ours	no	Qwen3.5-9B	87.2 / 66.9	83.1 / 57.1	91.0%
LeapQuant, 4-bit	no	Qwen3.5-35B-A3B	91.5 / 75.6	59.2 / 37.4	57.8%
Ours	no	Qwen3.5-35B-A3B	92.8 / 74.5	84.4 / 65.5	89.6%
STEPQuant, 4-bit budget	yes	Qwen3.8-27B	87.7 / 85.3	86.3 / 86.4	99.8%
Ours	no	Qwen3.8-27B	96.4 / 83.2	96.4 / 84.2	100.6%

Table E.1. Accuracy retention, the mean of the two tasks divided by the same mean for the FP32 state of the same study. LeapQuant (Table 1) and STEPQuant (Table 1) evaluate AIME 2026, we evaluate AIME 2025 and 2026.

LeapQuant is the closer comparison, since it also needs no calibration. At 4 bits it retains about 60% of its FP32 accuracy on the two Qwen3.5 models, against about 90% for our state. STEPQuant and our state both retain almost all of their FP32 accuracy on Qwen3.8-27B, but STEPQuant does so with calibration data and a mixed-precision layout, and in fewer bits per element, since rounding after every token needs no update buffer.